

Análisis del sistema de recompensas MiMcDonald’s Guatemala

1. Resumen de la situación

Conclusión principal. MiMcDonald’s puede modelarse como un sistema transaccional de lealtad cuyo activo central es un libro de movimientos de puntos. La app, el POS y los canales digitales identifican al cliente; los servicios de órdenes y reglas validan la compra; el libro registra créditos, débitos y vencimientos; y un pipeline separado transforma los eventos operacionales en datos analíticos. Esta separación permite mantener la compra disponible aunque fallen procesos de inteligencia de negocio o aprendizaje automático.

El programa público permite acumular y canjear puntos en Guatemala mediante la app y canales participantes. Sus términos establecen registro de mayores de edad, identificación previa al pago, códigos de cliente dinámicos, acumulación, límites diarios, vencimientos y restricciones de transferencia. El diagnóstico detecta cuatro necesidades prioritarias: una fuente única de reglas, idempotencia por orden, trazabilidad completa mediante un ledger y contratos de calidad para el pipeline.

La optimización propuesta utiliza eventos para desacoplar la operación del análisis. Las transacciones se conservan en una zona cruda, se limpian y validan mediante ETL y se cargan en un modelo dimensional. Sobre este almacén se plantea un modelo de riesgo de abandono que produzca segmentos accionables, sujeto a medición mediante experimentos controlados y a controles de privacidad, sesgo, deriva y auditoría.

2. Funcionamiento público del programa

2.1. Registro e identificación

El usuario mayor de edad crea una cuenta en la app y queda incorporado al programa. La cuenta se asocia con un correo electrónico y puede incluir un teléfono. La app presenta una tarjeta digital con código QR y código alfanumérico. Para atribuir una compra, el cliente debe identificarse antes del pago; los puntos no se acreditan retroactivamente.

2.2. Canales y restricciones

La acumulación pública cubre AutoMac, mostrador, pantallas digitales, McCafé, centros de postres y McDelivery dentro de la aplicación. El Call Center y plataformas externas de entrega quedan excluidos. La atribución y el canje requieren conectividad; si el sistema del restaurante está fuera de línea, la operación de lealtad puede suspenderse aunque la venta comercial continúe.

2.3. Reglas relevantes para la POC

Regla	Valor
Acumulación	10 puntos por Q1
Tope de acumulación	1,000 puntos diarios
Vigencia	365 días desde la generación
Canje diario	Hasta 15,000 puntos
Ofertas por pedido	Máximo 3, salvo excepción
Transferencia	No permitida
Canal	Solo Guatemala y canales participantes

3. Análisis funcional

3.1. Arquitectura de puntos

- El cliente presenta su QR o usa una sesión autenticada antes del pago.
- El canal envía el token del cliente y el identificador de la orden al punto de entrada del sistema.
- El servicio de identidad resuelve el token a un customer_id válido.
- El servicio de órdenes confirma que la venta existe, está pagada y no fue procesada antes.
- El servicio de lealtad solicita al motor de reglas la cantidad elegible.
- El motor aplica tasa, exclusiones, promociones y tope diario.

- 7. El ledger registra un crédito inmutable con order_id y fecha de expiración.
- 8. El sistema publica POINTS_EARNED y devuelve el nuevo saldo a la app.

3.2. Canje de una recompensa

- 1. La app consulta el catálogo y muestra únicamente recompensas vigentes para el canal y horario.
- 2. El cliente selecciona un reward_id y solicita el canje.
- 3. El sistema valida identidad, saldo, límite diario, número de ofertas y disponibilidad.
- 4. Se reserva el saldo para impedir canjes concurrentes sobre los mismos puntos.
- 5. Al confirmar el pedido, el ledger registra un débito y la orden incorpora el producto recompensado.
- 6. El evento REWARD_REDEEMED alimenta auditoría y analítica.

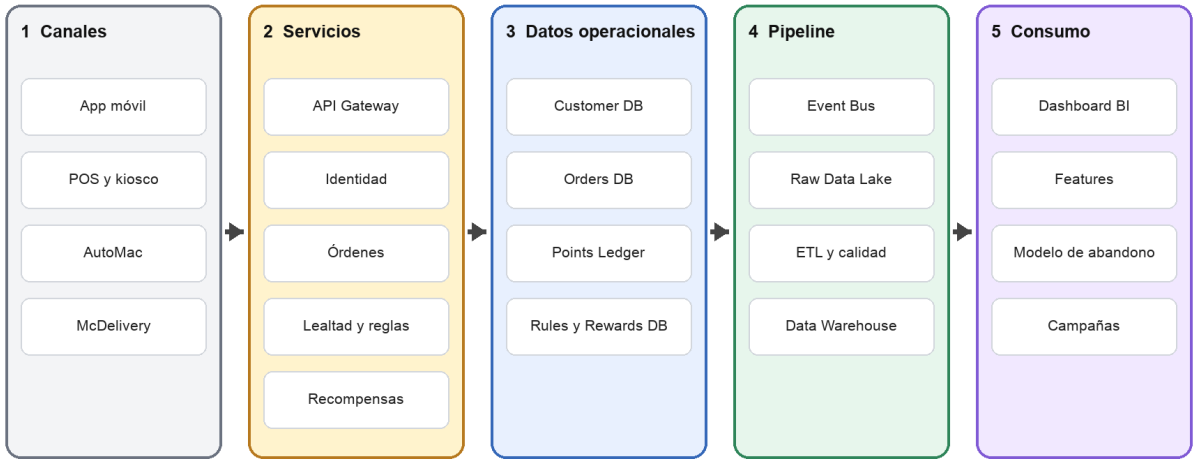
3.3. Vencimiento y saldo

El saldo no debe ser un campo editable sin historial. Se calcula como la suma de créditos vigentes menos débitos y ajustes. Cada lote de puntos conserva su fecha de generación y vencimiento; para canjear se recomienda consumir primero los créditos que expiran antes. Un proceso diario identifica lotes vencidos, registra POINTS_EXPIRED y actualiza las vistas de saldo.

4. Arquitectura de datos

Arquitectura conceptual de datos de MiMcDonald's

Flujo operacional en tiempo real y flujo analítico desacoplado



Línea principal: transacción y eventos | Extracciones batch o CDC complementan el flujo al almacén analítico

La separación entre operación y analítica evita que un fallo del pipeline bloquee una compra o un canje.

4.1. Capas y canales

App, POS, kiosco, AutoMac y McDelivery producen solicitudes de identidad, órdenes, pagos y canjes. Cada solicitud debe incluir un identificador de correlación que permita rastrearla a través de los servicios.

4.2. Servicios transaccionales

El API Gateway centraliza el enrutamiento y la autenticación de cada solicitud proveniente de los canales. Identidad valida sesiones y credenciales; Órdenes gestiona el ciclo de vida del pedido; Lealtad y reglas aplica la lógica de acumulación y canje de puntos; Recompensas administra el catálogo de beneficios. Separar estos servicios permite escalar y depurar cada uno de forma independiente antes de persistir los datos.

4.3. Datos operacionales

Customer DB mantiene cuentas; Orders DB conserva ventas y estados; Points Ledger es la fuente de verdad de créditos y débitos; Rules DB versiona reglas; Rewards DB administra el catálogo. Separar estas responsabilidades reduce acoplamiento y permite auditar cambios sin reconstruir la historia desde un saldo mutable.

4.4. Pipeline

Los servicios publican eventos como ORDER_PAID, POINTS_EARNED, POINTS_REDEEMED y POINTS_EXPIRED. Un bus de eventos desacopla productores y consumidores. La zona raw preserva el mensaje original; el ETL normaliza tipos, elimina duplicados técnicos, verifica claves y aplica reglas de calidad. Las extracciones batch o CDC complementan el stream para reconciliar datos operacionales.

4.5. Consumo

El Data Warehouse utiliza tablas de hechos para órdenes, movimientos y canjes, más dimensiones conformadas para cliente, restaurante, recompensa, canal y fecha. Los dashboards y modelos reciben datos gobernados y no acceden directamente a bases operacionales.

5. Pipeline

5.1. Extracción

La POC leerá archivos CSV o JSON sintéticos que representan clientes, órdenes, movimientos y recompensas. En producción, la fuente principal serían eventos y la reconciliación periódica de tablas operacionales.

5.2. Limpieza

- Convertir fechas a UTC y conservar la zona horaria del negocio para límites diarios.
- Normalizar canales y estados a catálogos controlados.
- Eliminar duplicados técnicos por event_id sin eliminar compras legítimas repetidas.
- Rechazar montos negativos, claves vacías y relaciones inexistentes.
- Comparar puntos calculados con la versión de regla aplicable en la fecha del evento.
- Enviar registros inválidos a cuarentena con código de error y trazabilidad.

5.3. Transformación y carga

El pipeline deriva métricas, resuelve claves dimensionales y carga de forma incremental. El watermark de carga debe permitir reanudar el proceso sin duplicar hechos. Las actualizaciones de dimensiones sensibles al tiempo, como estado de recompensa o restaurante, necesitan vigencia efectiva.

5.4. Controles de calidad

Control	Criterio de aceptación
Unicidad de eventos	event_id único; duplicados enviados a registro de observación.
Integridad de órdenes	100 % de movimientos EARN asociados con orden pagada válida.
Saldo	Ningún cliente con saldo disponible negativo.
Reglas	Diferencia cero entre puntos esperados y puntos registrados, salvo ajustes autorizados.
Oportunidad	Eventos críticos disponibles en el warehouse dentro del SLA acordado.
Reconciliación	Totales diarios del ledger coinciden con créditos, débitos y expiraciones cargados.

6. Propuesta de integración de aprendizaje automático

6.1. Caso de uso

Se propone un modelo de clasificación para estimar el riesgo de que un cliente deje de comprar. El objetivo no es enviar más promociones a todos, sino priorizar clientes con probabilidad alta de abandono y una intervención económicamente justificable. Una definición inicial de la etiqueta es ausencia de compras durante 60 días posteriores a una ventana de observación; debe ajustarse con datos reales de frecuencia y estacionalidad.

6.2. Variables

Grupo	Variables propuestas
Recencia	Días desde la última compra y desde el último canje.
Frecuencia	Órdenes en 30, 60 y 90 días; variación contra periodo anterior.
Valor	Gasto total, ticket promedio y tendencia de gasto.
Lealtad	Saldo, puntos próximos a vencer, acumulación y tasa de redención.
Canal	Canal preferido, diversidad de canales y restaurante frecuente.
Campañas	Ofertas vistas, utilizadas y tiempo hasta conversión.
Grupo	Variables propuestas

6.3. Pipeline ML

- Extraer una tabla de observación desde el Data Warehouse sin información futura.
- Construir features reproducibles y registrar la fecha de corte.
- Separar entrenamiento, validación y prueba de forma temporal.
- Entrenar una línea base interpretable, por ejemplo regresión logística, y compararla con gradient boosting.
- Evaluar PR-AUC, recall en el segmento objetivo, lift, calibración y costo por intervención.
- Registrar versión de datos, código, hiperparámetros y modelo.
- Calcular scores periódicos y enviarlos al Campaign Engine, no directamente al cliente.
- Medir impacto mediante prueba A/B contra una política de control.

6.4. Evaluación empresarial

La precisión predictiva no demuestra valor comercial. La métrica final debe comparar margen incremental con costo de incentivos y comunicaciones. El modelo solo se aprueba si el grupo tratado presenta una mejora estadística y económicamente relevante en retención o frecuencia frente al control.

6.5. Riesgos y gobierno

- Excluir correo, teléfono y otros identificadores directos de las variables de entrenamiento.
- Revisar desempeño por segmentos para detectar diferencias injustificadas.
- Monitorear deriva de variables, calibración, tasa de respuesta y costo de campaña.
- Mantener reglas de exclusión para clientes sin consentimiento de marketing.
- Permitir revisión, suspensión y reversión del modelo.
- Documentar propósito, limitaciones, periodo de datos y responsables siguiendo prácticas de gestión de riesgo de NIST.

7. Código/procedimiento de una prueba funcional (POC) del sistema de puntos

Ver link de Github

8. Conclusiones

- El libro de movimientos es la fuente de verdad del sistema; un saldo mutable sin historial no satisface auditoría, vencimientos ni compensaciones.
- La identificación previa al pago y la idempotencia por orden son controles funcionales indispensables para evitar pérdidas y duplicación de puntos.
- El motor de reglas debe ser configurable y versionado, especialmente ante la inconsistencia documental encontrada sobre la tasa de acumulación.
- La arquitectura orientada a eventos separa la experiencia de compra de analítica y ML, mejorando disponibilidad y capacidad de evolución.
- El pipeline debe conservar datos crudos, validar contratos y reconciliarse con sistemas operacionales antes de alimentar decisiones.
- El modelo de abandono solo aporta valor si se evalúa con separación temporal, métricas de negocio y experimentos controlados.
- La privacidad, el consentimiento y la supervisión del modelo deben diseñarse desde el inicio, no añadirse después del despliegue.

9. Referencias

- Industria de Hamburguesas, S. A. (2025). Términos y condiciones de la aplicación McDonald's Guatemala, actualización del 18 de agosto de 2025. <https://mcdonaldsdigital.com/GML/public/app/gt/terminos-y-condiciones>
- 2. McDonald's Guatemala. (2026). MiMcDonald's: programa de lealtad y catálogo de recompensas. <https://www.mi.mcdonalds.com.gt/>
- 3. McDonald's Guatemala. (2026). ¿Cómo realizo el canje de puntos? Página de ayuda. <https://mcdonaldsguatemalahelp.zendesk.com/hc/es-gt/articles/36748791684635--C%C3%B3mo-realizo-el-canje-de-puntos>
- 4. Microsoft. (2026). Event-driven architecture style. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven>

Link de Github: https://github.com/ArielaMishaanCohen/Caso_2